
GenAI Erfahrungsbericht G4T1

Dieser Bericht dokumentiert den Einsatz von generativer KI im Rahmen des G4T1-Projekts.

Verwendete GenAI-Tools waren ChatGPT und Claude, insbesondere für Code-Reviews und Refactoring-Vorschläge.

Code-Reviews

Beispiel: ProductController – Exception Handling

Prompt:

„Analysiere den ProductController auf potenzielle Verbesserungen im Exception Handling. Identifiziere Code-Duplikationen und schlage eine konsistente Fehlerbehandlung vor.“

Ergebnis der Analyse:

Die Analyse ergab mehrere Verbesserungspotenziale. Es wurden wiederholte try-catch-Blöcke mit identischem Exception-Handling identifiziert. Zudem wurde festgestellt, dass eine sehr generische Exception-Behandlung ohne spezifische Fehlertypen verwendet wurde. Außerdem wurde eine `ResponseStatusException` gefangen und erneut geworfen, was als redundant bewertet wurde.

Vorgeschlagene Lösung:

Die KI schlug vor, das Exception-Handling in eine zentrale Klasse mit `@RestControllerAdvice` auszulagern.

@RestControllerAdvice

```
public class ProductControllerAdvice {
    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<ErrorResponse> handleResponseStatusException(
        ResponseStatusException ex) {
        return ResponseEntity.status(ex.getStatusCode())
            .body(new ErrorResponse(ex.getReason()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("Unexpected error: " + ex.getMessage()));
    }
}
```

Umsetzung:

Die Controller-Methoden wurden vereinfacht und das Exception-Handling vollständig in einen zentralen `@RestControllerAdvice` ausgelagert.

Testfallideen

Beispiel: CartService – Edge Cases

Verwendeter Prompt:

„Generiere Testfälle für den CartService, insbesondere für Edge Cases und Fehlerszenarien. Berücksichtige Transaktionssicherheit und Race Conditions.“

Beispiel für einen Testfall:

```
@Test
@Transactional
void testAddItemToCart_InsufficientStock() {
    Product product = createProductWithStock(5);
    CartItem cartItem = new CartItem();
    cartItem.setProductId(product.getId());
    cartItem.setQuantity(10);

    assertThrows(ResponseStatusException.class, () -> {
        cartService.addItemToCart(cartItem);
    });

    Cart cart = cartService.getCart();
    assertTrue(cart.getItems().isEmpty());
}
```

Umsetzung:

Mehr als 7 Testfälle wurden auf Basis dieser Vorschläge implementiert.

Zusammenfassung und Erkenntnisse

Der Einsatz von GenAI führte zu einer deutlichen Zeitersparnis bei der Identifikation von Code-Duplikationen und potenziellen Problemen. Zudem konnte die Code-Qualität durch systematische Analysen von Edge Cases und Fehlerszenarien verbessert werden.

Herausforderungen zeigten sich insbesondere beim begrenzten Kontextverständnis der KI, wodurch teilweise mehrere Iterationen notwendig waren. Außerdem mussten alle Vorschläge kritisch überprüft werden, da manche Ansätze für das Projekt zu komplex gewesen wären.